

Image Recognition

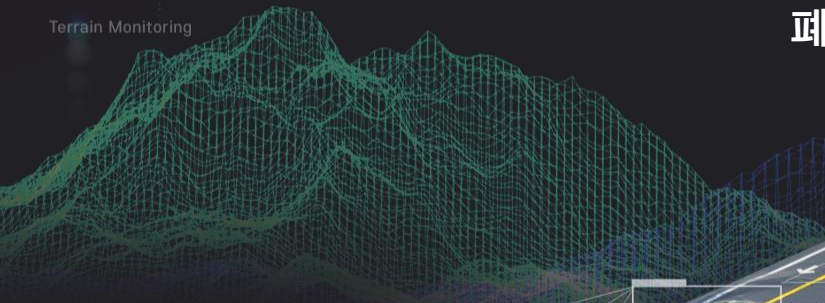


강원혁신플랫폼

컴퓨터 비전

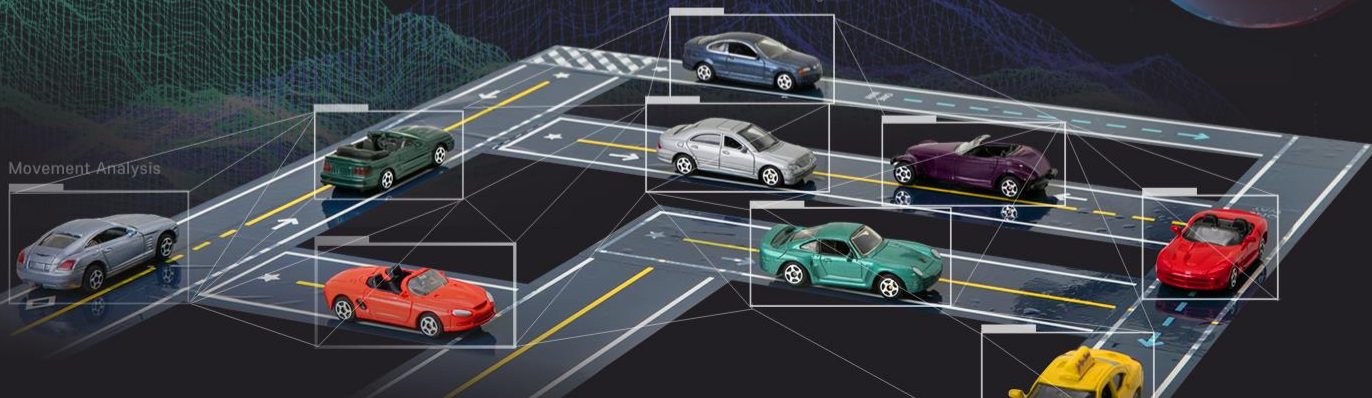
◆
폐렴 진단 이진 분류

Terrain Monitoring



Autonomous Driving

Movement Analysis



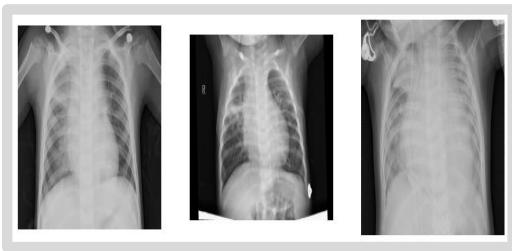
폐렴 진단하기

정상적인 사람과 폐렴에 걸린 사람의 흉부 X-Ray 사진 5,216장을 학습시켜서 새로운 사람의 흉부 X-Ray 사진을 입력했을 때 그것이 정상인지 폐렴인지를 분류해 내는 모델을 만들어 보자.

이진 이미지 분류(Image Binary Classification)

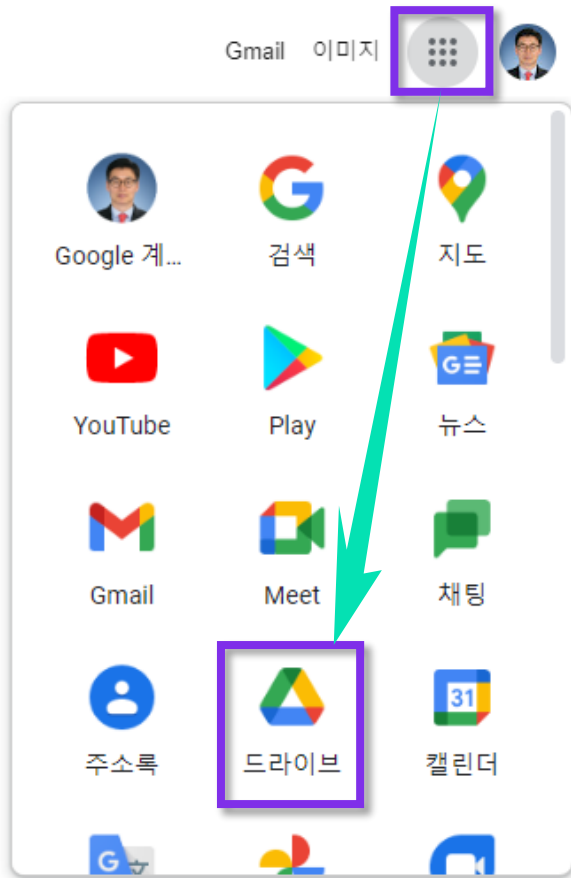
- 인공신경망(딥러닝)의 세부 알고리즘은 차차 배우기로 하고, 정상적인 사람과 폐렴에 걸린 사람 분류의 전체과정과 실습을 통하여 학습해 보자.
- 실습은 구글 Colab을 이용한다.

의료 영상(Medical Image) 분석



예 X-Ray, CT, MRI 등

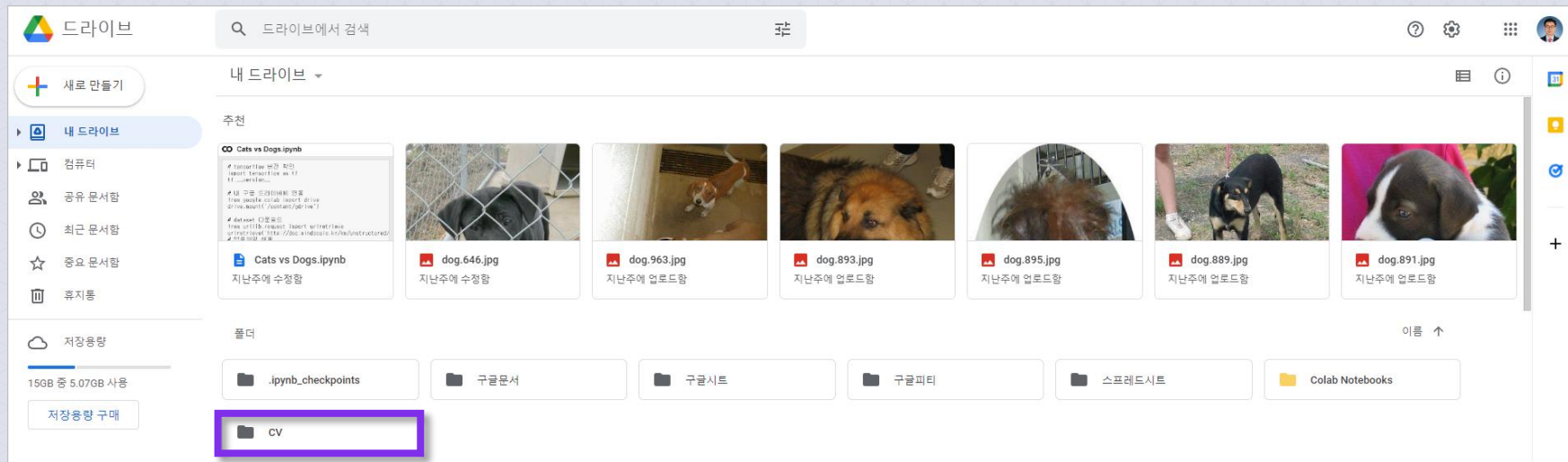
① 실습 파일 복사하기



- 구글 Colab을 이용하기 위해
구글 크롬 브라우저에서 그림과 같이
구글 드라이브를 클릭한다.

① 실습 파일 복사하기

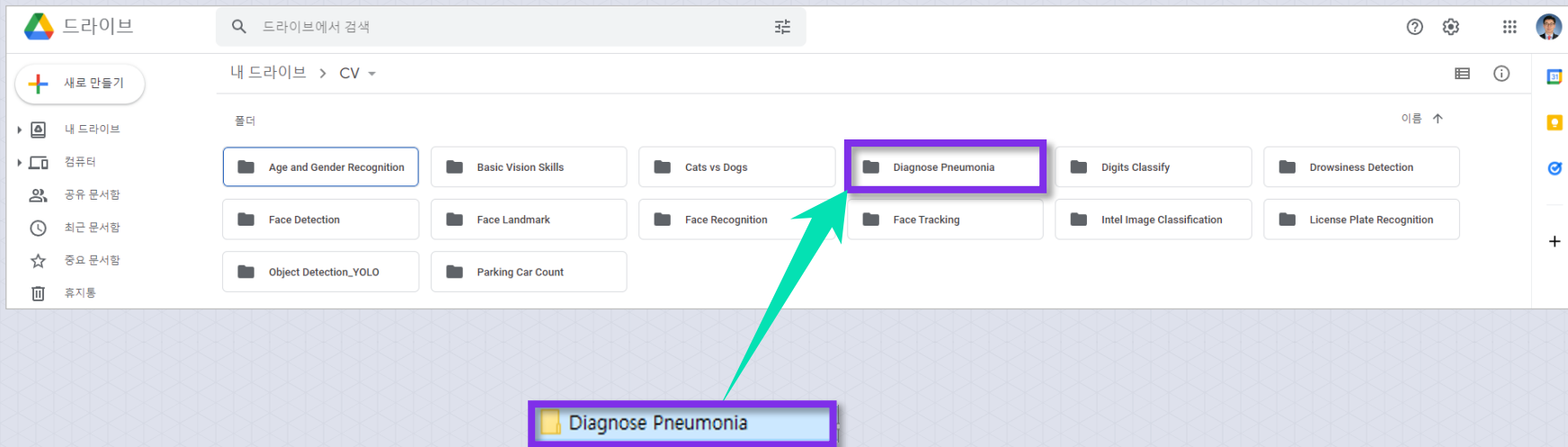
구글 드라이브에서 CV 폴더를 클릭한다.



CV 폴더는 개와 강아지 분류 문제에서 생성된 폴더이며,
CV 폴더가 없는 경우 직접 생성해도 됨(CV 폴더가 최상위 폴더)

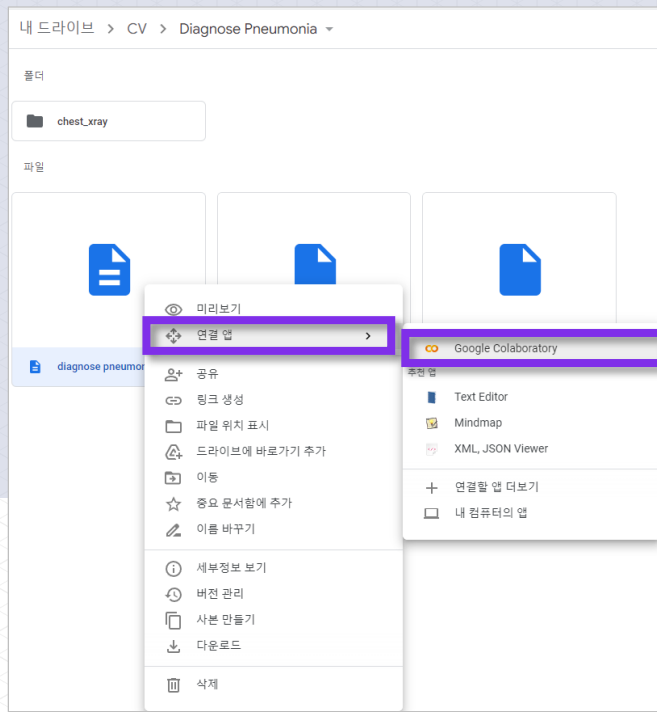
① 실습 파일 복사하기

- 구글 드라이브 > CV > **Diagnose Pneumonia** 폴더를 클릭한다.
- 강의 실습 자료를 다운로드하여 [**Diagnose Pneumonia**] 폴더를 구글 드라이브 CV 폴더 아래에 복사한다.



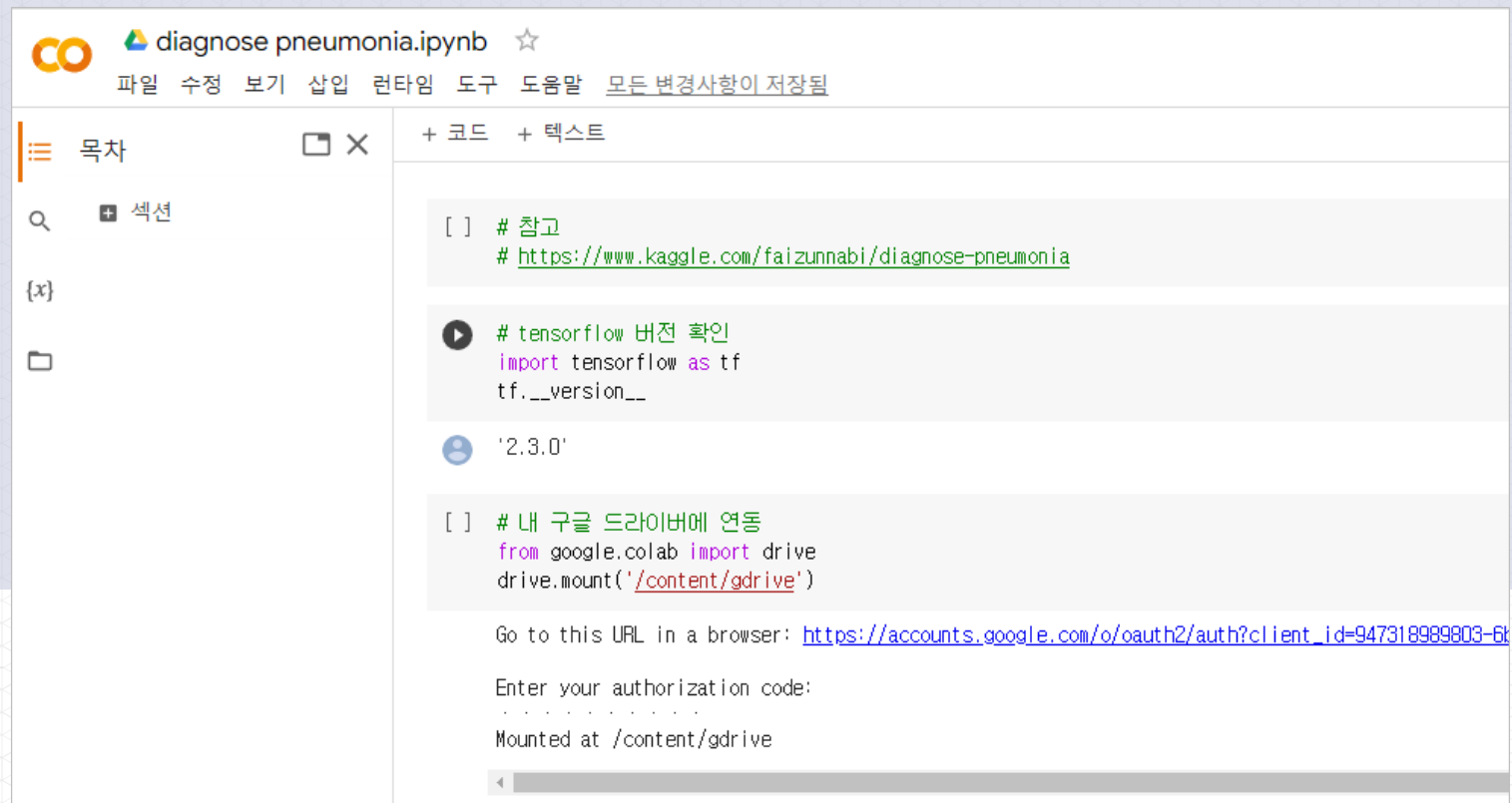
① 실습 파일 복사하기

- 아래의 그림의 **Diagnose Pneumonia** 폴더에서 **diagnose pneumonia.ipynb**를 선택 후 오른쪽 마우스를 클릭하여 연결 앱 > **Google Colaboratory**을 클릭한다.



① 실습 파일 복사하기

🔄 아래 그림과 같이 구글 Colab 노트북 화면이 보이면 실습 준비가 완료된 것이다.



② Tensorflow의 버전 확인하기

🟢 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



```
# tensorflow 버전 확인  
import tensorflow as tf  
tf.__version__
```

```
'2.8.2'
```

실행 결과 tensorflow 버전이 '2.8.2'인 것을 알 수 있다.

③ 구글 드라이브에 연동하기

🔄 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.



```
# 내 구글 드라이브에 연동
```

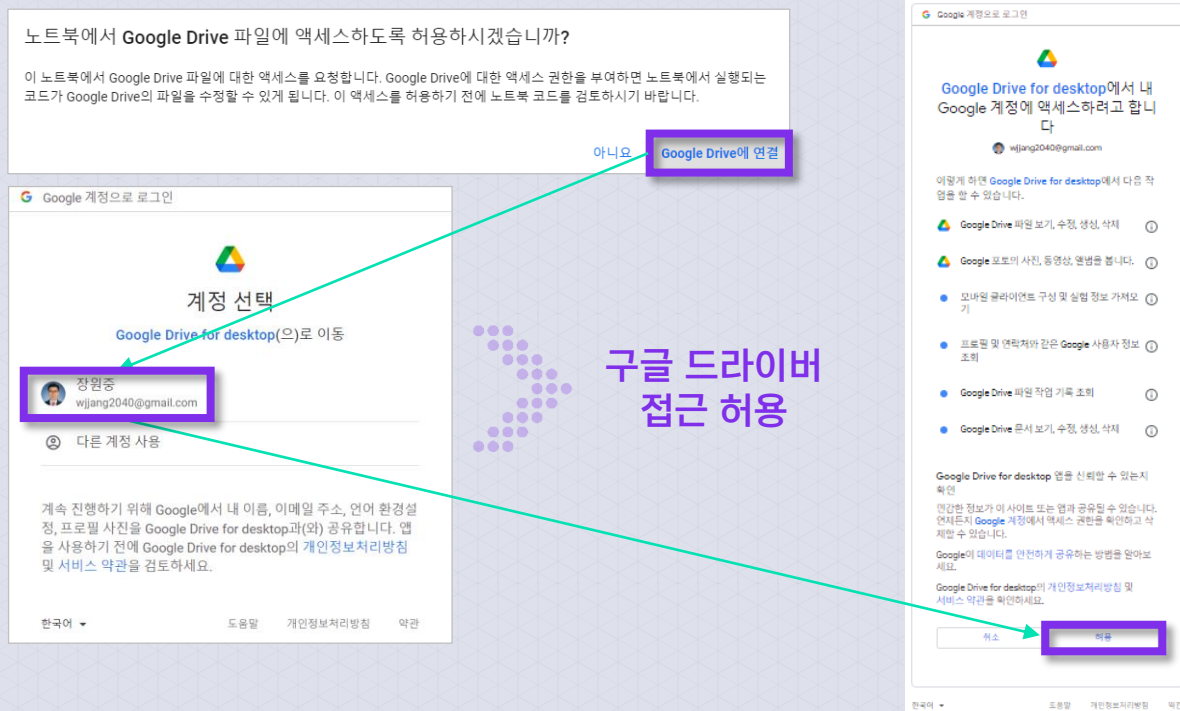
```
from google.colab import drive
```

```
drive.mount('/content/gdrive')
```

```
Drive already mounted at /content/gdrive; to attempt to forcibly remount, call drive.mount("/content/gdrive", force_remount=True).
```

③ 구글 드라이브에 연동하기

- ⦿ [Google Drive에 연결] > [계정 선택] > [구글 드라이브 접근 허용] 메뉴를 선택한다.
- ⦿ gdrive에 연동되면 구글 드라이브에 저장된 데이터를 활용할 수 있다.



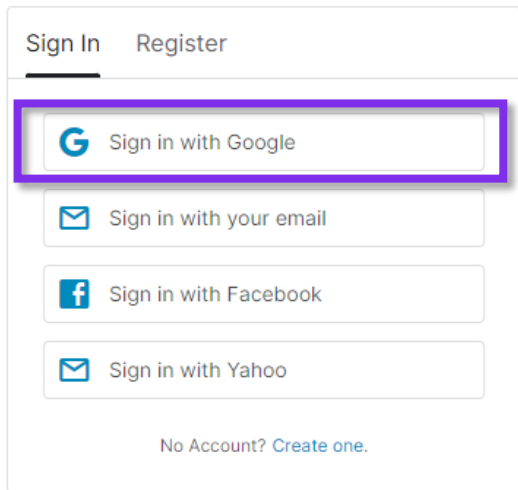
④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

- 1 ➤ 모델의 학습과 예측에 사용할 흉부 X-Ray 데이터 셋이 Kaggle에 있으므로, 구글 Colab에서 Kaggle에 연동하여 데이터 셋을 다운로드 받는다.
- 2 ➤ Kaggle에서 데이터 셋을 다운 받기 위해서는 Kaggle에 우선 회원가입이 되어 있어야 한다.
- 3 ➤ 흉부 X-Ray 데이터 셋을 Kaggle에서 직접 다운로드 받는 절차는 반드시 필요하다.
 - ⌚ 흉부 X-Ray 데이터 셋의 용량이 너무 커서 파일첨부가 어렵기 때문임
 - ⌚ 여기서 Kaggle 회원가입 절차는 생략함

④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

kaggle

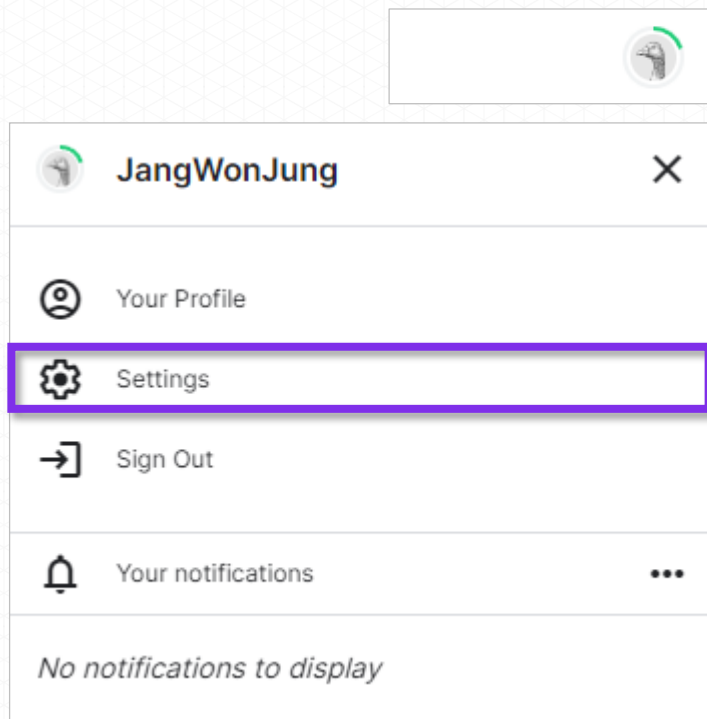
- 🔄 Kaggle에 연동하기 위해서 API Token을 받아와야 하는데 일단 kaggle.com로 접속한다.
- 🔄 그림과 같은 창이 뜨면 [로그인](#)을 한다.



The image shows the Kaggle login/register interface. At the top, there are two tabs: 'Sign In' and 'Register'. Below the tabs, there are four buttons for signing in: 'Sign in with Google' (highlighted with a purple border), 'Sign in with your email', 'Sign in with Facebook', and 'Sign in with Yahoo'. At the bottom, there is a link that says 'No Account? Create one.'

④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

- 로그인 하면 우측 상단에 오리 아이콘을 클릭하고, Settings를 선택한다.



④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

- 아래 화면의 중간쯤 스크롤을 내리면 새로운 API를 생성할 수 있는 [Create New Token] 버튼이 있다.
- 이 버튼을 누르면 kaggle.json 파일이 다운로드 된다.

Settings

Control over your Kaggle account and all communications

Account Notifications

Your email address
wjang2040@gmail.com
Change email

Your username
jangwonjung (not editable)

Your account number
7160486

Phone verification

Your account is not verified. Verifying your account with a phone number allows you to do more on Kaggle, and helps prevent spam and other abuse.

Phone verify

API

Using Kaggle's beta API, you can interact with Competitions and Datasets to download data, make submissions, and more via the command line. [Read the docs](#)

Ensure kaggle.json is in the location ~/.kaggle/kaggle.json to use the API.

Dismiss

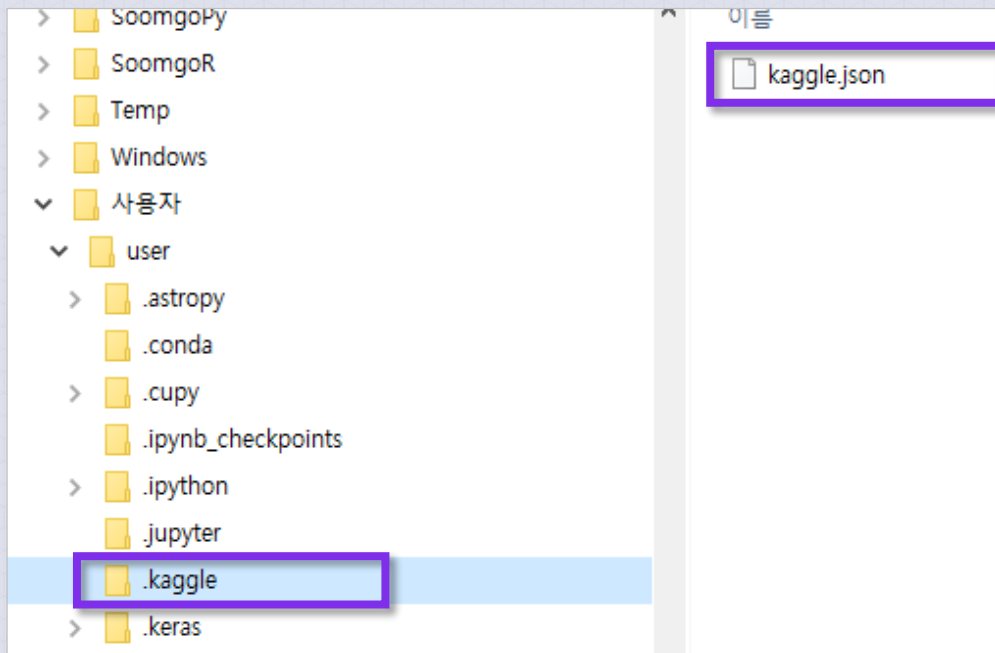
Create New Token

Expire Token

④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

아나콘다(Anaconda)를 사용하는 경우

- ➡ [사용자 계정] 폴더 > .kaggle 폴더 아래에 kaggle.json 파일을 복사한다.
- ➡ 아래의 경우 사용자 계정이 'user'인 경우의 예시이다.



④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

구글 Colab인 경우

- ➊ Kaggle API(kaggle 라이브러리)를 설치하고, **kaggle.json** 파일을 VM에 **업로드** 한다.
- ➋ 구글 Colab 노트북 화면에서 **실행 버튼**을 클릭하면 된다.
- ➌ 구글 Colab 노트북 화면에서 **kaggle.json** 파일을 **VM에 업로드** 하는 경우 아래 화면의 **[파일 선택]**을 **클릭**하여 kaggle.json 파일을 업로드 해 준다.



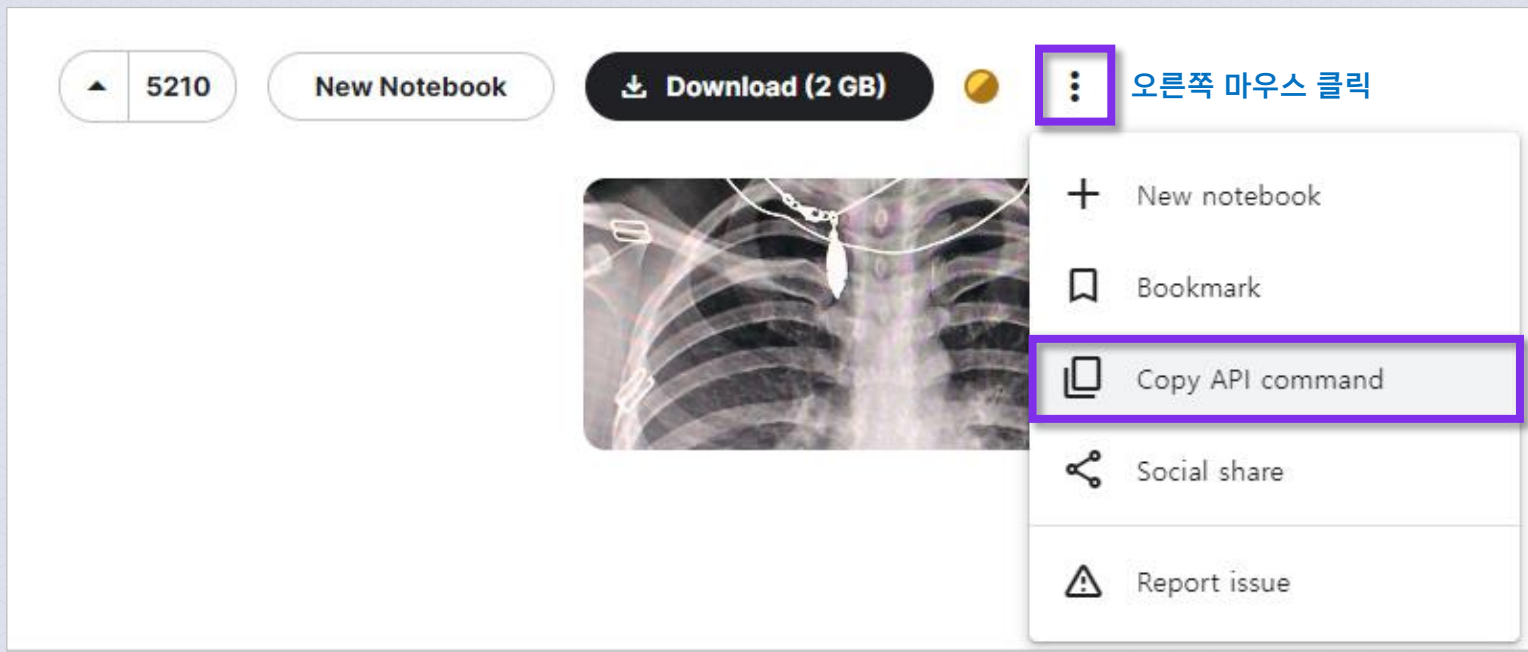
```
# 1.먼저 본인의 Kaggle 계정에서 API Token 을 다운받습니다.(kaggle.json)
# 2.Kaggle API를 설치합니다.
!pip install kaggle
# 3.kaggle.json 파일을 VM에 업로드 합니다.
from google.colab import files
files.upload()
```

Requirement already satisfied: kaggle in /usr/local/lib/python3.6/dist-packages (1.5.6)
Requirement already satisfied: six>=1.10 in /usr/local/lib/python3.6/dist-packages (from kaggle) (1.15.0)
Requirement already satisfied: python-slugify in /usr/local/lib/python3.6/dist-packages (from kaggle) (4.0.1)
Requirement already satisfied: requests in /usr/local/lib/python3.6/dist-packages (from kaggle) (2.23.0)
Requirement already satisfied: python-dateutil in /usr/local/lib/python3.6/dist-packages (from kaggle) (2.8.1)
Requirement already satisfied: tqdm in /usr/local/lib/python3.6/dist-packages (from kaggle) (4.41.1)
Requirement already satisfied: certifi in /usr/local/lib/python3.6/dist-packages (from kaggle) (2020.6.20)
Requirement already satisfied: urllib3<1.25,>=1.21.1 in /usr/local/lib/python3.6/dist-packages (from kaggle) (1.24.3)
Requirement already satisfied: text-unidecode>=1.3 in /usr/local/lib/python3.6/dist-packages (from python-slugify->kaggle) (1.3)
Requirement already satisfied: chardet<4,>=3.0.2 in /usr/local/lib/python3.6/dist-packages (from requests->kaggle) (3.0.4)
Requirement already satisfied: idna<3,>=2.5 in /usr/local/lib/python3.6/dist-packages (from requests->kaggle) (2.10)

파일 선택 선택된 파일 없음 Upload widget is only available when the cell has been executed in the current browser session. Please rerun this cell to enable.
saving kaggle.json to kaggle.json
{'kaggle.json': b'{"username": "briansong", "key": "247d332863eec5370ba5dc718656d51e"}'}

④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

- ➡ Kaggle의 **흉부 X-Ray 데이터** 셋을 분석 노트북에 **다운로드** 받기 위해 [https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia]로 접속한다.
- ➡ 위 URL로 접속하면 아래 화면이 출력되고, 이 화면에서 **Copy API command**를 **선택**한다.



④ Kaggle에 연동하여 흉부 X-Ray 데이터 다운받기

- [Copy API command]를 [Ctrl + v]로 셀에 붙여 넣은 다음 명령어를 실행하면 흉부 X-Ray 데이터 셋이 다운로드 된다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



```
# dataset URL : https://www.kaggle.com/paultimothymooney/chest-xray-pneumonia  
# 7.Kaggle 에서 chest-xray dataset을 다운 받기 위해 API Command 를 복사해서 가져옵니다.  
!kaggle datasets download -d paultimothymooney/chest-xray-pneumonia
```



```
Downloading chest-xray-pneumonia.zip to /content  
100% 2.29G/2.29G [00:28<00:00, 50.7MB/s]  
100% 2.29G/2.29G [00:29<00:00, 84.8MB/s]
```

실행 결과 ‘chest-xray-pneumonia.zip’ 이름으로 VM에 다운로드 된다.

⑤ 압축 파일(zip) 해제하기

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



압축파일 해제

```
from zipfile import ZipFile
```

```
with ZipFile('chest-xray-pneumonia.zip') as z:  
    z.extractall()
```

- 실행 결과 VM에 chest_xray 폴더가 생성된다.
- chest_xray > chest_xray > test
 - > train > NORMAL
 - > PNEUMONIA
 - > val

⑥ 모델의 학습과 예측을 위해 필요한 패키지와 모델을 불러오기

🔄 구글 Colab 노트북 화면에서 **실행 버튼**을 **클릭**하면 된다.



필요한 패키지와 모듈을 불러옴

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import cv2
import os
import random
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D,Dense,Flatten,Dropout,MaxPooling2D
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras.callbacks import EarlyStopping, ModelCheckpoint
```

페렴진단 학습에 필요한 패키지를 읽어온다(메모리에 로딩한다).

⑦ 훈련 데이터 셋 로드하기

- 훈련용 데이터 셋(Train dataset)에서 정상적인 사람과 폐렴에 걸린 사람의 폐 사진을 각각 가져온다.
- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



train dataset 에서 정상적인 사람과 폐렴에 걸린 사람의 폐사진을 각각 가져옵니다.

```
total_images_train_normal = os.listdir('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/train/NORMAL/')  
total_images_train_pneumonia = os.listdir('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/train/PNEUMONIA/')
```

- 위와 같이 chest_xray > train 폴더에 정상적인 사람과 폐렴에 걸린 사람의 사진을 읽어온다.
- chest_xray > chest_xray > train > NORMAL
 > PNEUMONIA

⑦ 훈련 데이터 셋 로드하기

정상적인 사람의 폐 사진 샘플 6장 보여주기

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.



정상적인 사람의 폐사진 샘플을 보여줍니다.

```
sample_normal = random.sample(total_images_train_normal,6)  
f,ax = plt.subplots(2,3,figsize=(15,9))
```

```
for i in range(0,6):
```

```
    im = cv2.imread('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/train/NORMAL/'+sample_normal[i])
```

```
    ax[i//3,i%3].imshow(im)
```

```
    ax[i//3,i%3].axis('off')
```

```
f.suptitle('Normal Lungs')
```

```
plt.show()
```

위 코드에서 정상적인 사람의 폐 사진
전체에서 샘플 6장을 임의로 선택하여
출력한 것을 알 수 있음



⑦ 훈련 데이터 셋 로드하기

폐렴에 걸린 사람의 폐 사진 샘플 이미지 6장 보여주기

🔄 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 된다.

```
# 폐렴에 걸린 사람의 폐사진 샘플을 보여줍니다.
sample_pneumonia = random.sample(total_images_train_pneumonia, 6)
f, ax = plt.subplots(2, 3, figsize=(15, 9))

for i in range(0, 6):
    im = cv2.imread('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/train/PNEUMONIA/'+sample_pneumonia[i])
    ax[i//3, i%3].imshow(im)
    ax[i//3, i%3].axis('off')
f.suptitle('Pneumonia Lungs')
plt.show()
```

위 코드에서 폐렴에 걸린 사람의 폐 사진
전체에서 샘플 6장을 임의로 선택하여
출력한 것을 알 수 있음



⑧ 하이퍼파라미터 값 설정하기

🔄 구글 Colab 노트북 화면에서 **실행 버튼**을 **클릭**하면 된다.



```
# 하이퍼파라미터 설정
image_height = 150 # 이미지 높이
image_width = 150  # 이미지 넓이
batch_size = 10    # batch size
no_of_epochs  = 30  # training 횟수
```

위 코드에서 학습을 위한 **배치 크기 = 10**,
학습 횟수(no_of_epochs) = 30으로 설정한 것을 볼 수 있다.

다음은 **인공신경망 네트워크**를 구성하여 이미지 **분류**를 위한 **모델**을 만든다.

- CNN, Fully connected neural network, Relu, Sigmoid, MaxPooling, Cross entropy error, RMSprop, Accuracy, Dropout 알고리즘을 이용함

```
▶ # create model(모델을 만듭니다)
model = Sequential()
model.add(Conv2D(32, (3, 3), input_shape=(image_height, image_width, 3), activation='relu'))
model.add(Conv2D(32, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(Conv2D(64, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(Conv2D(128, (3, 3), activation='relu'))
model.add(MaxPooling2D(pool_size=(2, 2)))
model.add(Dropout(0.2))
model.add(Flatten())
model.add(Dense(units=128, activation='relu'))
model.add(Dropout(0.2))
model.add(Dense(units=1, activation='sigmoid'))
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

model.summary()
```

옆 코드와 같이
8층 인공신경망 모델을 생성

다음은 인공지능망 네트워크를 구성하여 이미지 분류를 위한 모델을 만든다.

인공지능망 네트워크 구성 결과

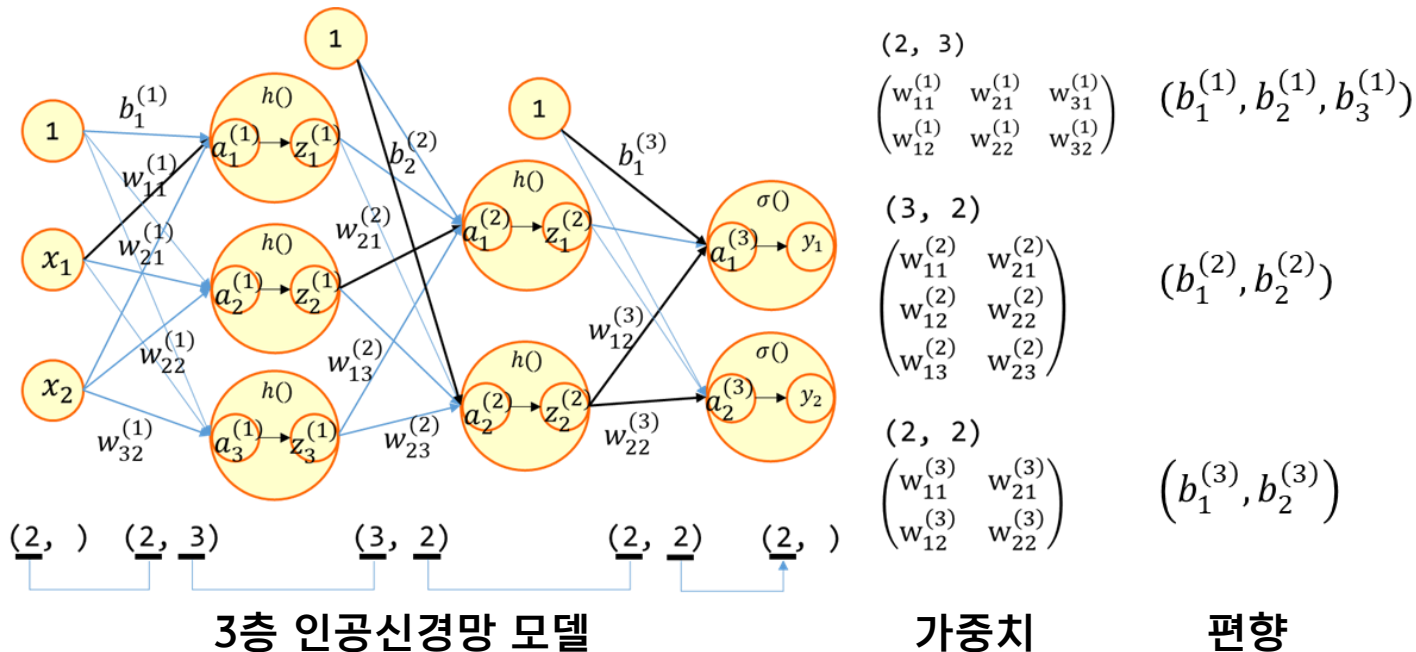


Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 148, 148, 32)	896
conv2d_1 (Conv2D)	(None, 146, 146, 32)	9248
max_pooling2d (MaxPooling2D)	(None, 73, 73, 32)	0
dropout (Dropout)	(None, 73, 73, 32)	0
conv2d_2 (Conv2D)	(None, 71, 71, 64)	18496
conv2d_3 (Conv2D)	(None, 69, 69, 64)	36928
max_pooling2d_1 (MaxPooling2D)	(None, 34, 34, 64)	0
dropout_1 (Dropout)	(None, 34, 34, 64)	0
conv2d_4 (Conv2D)	(None, 32, 32, 128)	73856
conv2d_5 (Conv2D)	(None, 30, 30, 128)	147584
max_pooling2d_2 (MaxPooling2D)	(None, 15, 15, 128)	0
dropout_2 (Dropout)	(None, 15, 15, 128)	0
flatten (Flatten)	(None, 28800)	0
dense (Dense)	(None, 128)	3686528
dropout_3 (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 1)	129
Total params: 3,973,665		
Trainable params: 3,973,665		
Non-trainable params: 0		

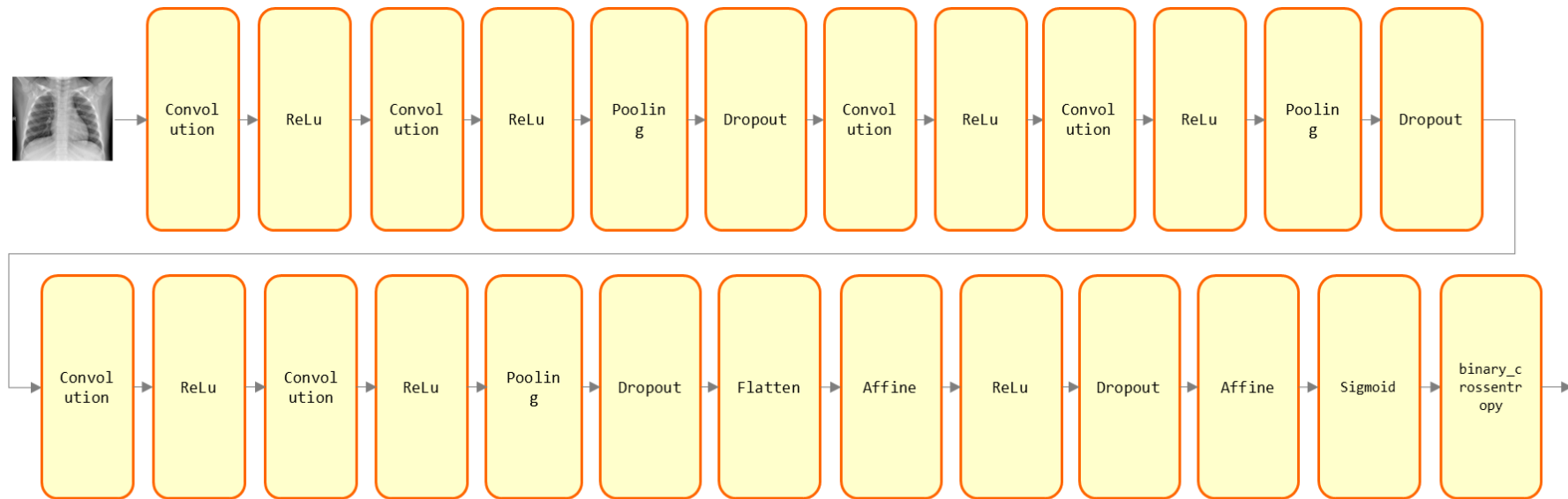
다음은 **인공신경망 네트워크**를 구성하여 이미지 분류를 위한 모델을 만든다.

(예시) **3층 인공신경망 모델**의 입력층, 은닉층, 출력층으로 신호 전달하는 과정



다음은 **인공신경망 네트워크**를 구성하여 이미지 분류를 위한 모델을 만든다.

8층 인공신경망 모델의 계층구조로 이루어진 네트워크 구조



인공신경망 네트워크 구성

⊕ 페렴 진단(학습)

다음은 ImageDataGenerator 클래스를 통해 `train_datagen` 인스턴스를 만든다.

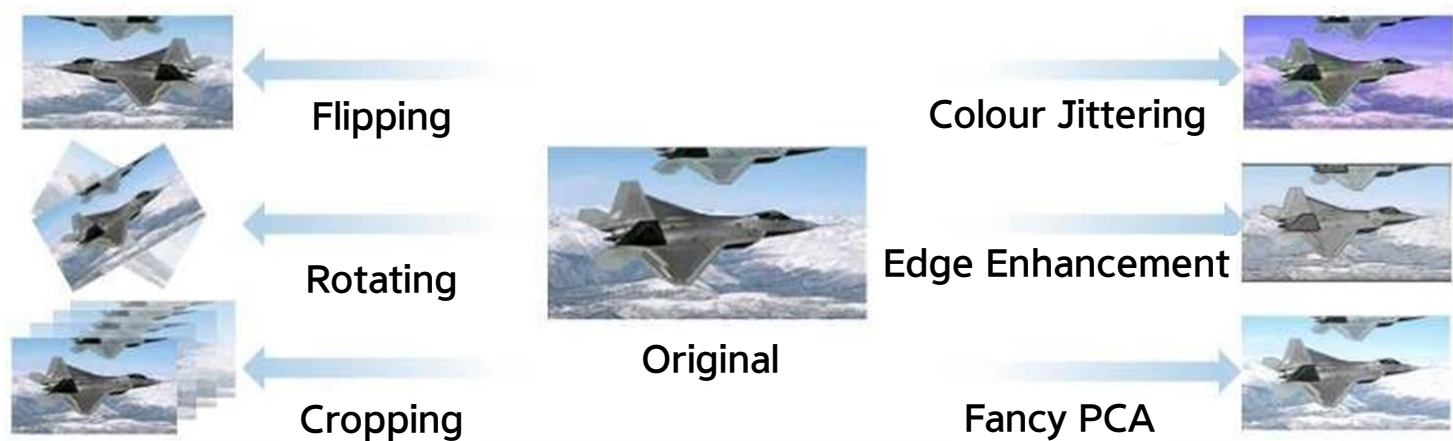
🔄 구글 Colab 노트북 화면에서 **실행 버튼**을 **클릭**하면 됨



```
1 # ImageDataGenerator 클래스를 통해 train_datagen 인스턴스를 만듭니다.  
2 train_datagen = ImageDataGenerator(rescale=1./255,      # 픽셀 값을 0~1 범위로 변환  
3                                   rotation_range=15,      # 15도까지 회전  
4                                   shear_range=0.2,        # 20%까지 기울임  
5                                   zoom_range=0.2          # 20%까지 확대  
6                                   )
```


다음은 ImageDataGenerator 클래스를 통해 `train_datagen` 인스턴스를 만든다.

- tensorflow.keras의 ImageDataGenerator 클래스는 데이터 증강(Data Augmentation)을 지원



여러 가지 데이터 증강 기법

⊕ 페렴 진단(학습)

다음은 ImageDataGenerator 클래스를 통해 `val_datagen` 인스턴스를 만든다.

- 구글 Colab 노트북 화면에서 실행 버튼을 클릭하면 됨



```
1 # ImageDataGenerator 클래스를 통해 val_datagen 인스턴스를 만듭니다
2 val_datagen = ImageDataGenerator(rescale=1./255)      # 픽셀 값을 0~1 범위로 변환
```

- tensorflow.keras의 ImageDataGenerator 클래스는 데이터 증강(Data Augmentation)을 지원
- 위 코드에서는 이미지 각 픽셀 값을 0~1 범위로 변환함



데이터 증강(Data Augmentation)



■ 이미지 회전과 무작위 변환을 적용해 훈련 데이터의 다양성을 증가시키는 기술

- ➡ 데이터의 양을 늘려 과적합 방지
- ➡ 훈련 데이터에만 적용
- ➡ 다양한 데이터 변형 상황을 대비

예) 숫자 6, 9 이미지와 같은 데이터일 경우 오히려 학습에 방해가 될 수 있으므로 데이터의 정보를 파악하고 적당히 변형시켜야 함(회전 10~20% 정도)

이미지 증강

예시

크기 조절, 회전, 밝기, 색깔 변경, 잘라내기



■ 데이터 증강(Data Augmentation)은 왜 필요할까?

1

▶ 과적합(오버피팅) 문제

- ❶ 이미지 분류 CNN 모델도 정형 데이터를 예측하는 일반적인 머신러닝 모델과 마찬가지로, 극복해야 할 문제점
- ❷ CNN 모델의 성능을 높이면서 오버피팅을 극복할 수 있는 가장 좋고 근본적인 해결책은 학습 데이터의 다양성을 늘리는 것임



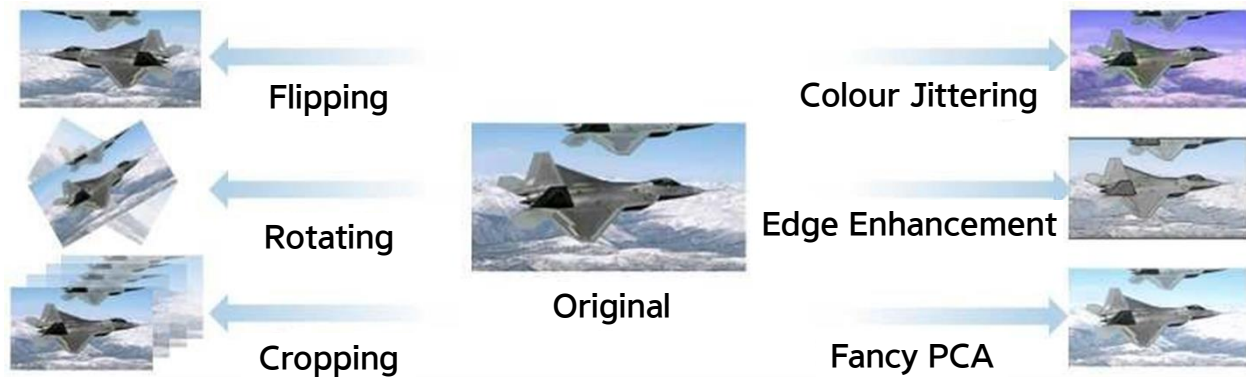
■ 데이터 증강(Data Augmentation)은 왜 필요할까?

2 이미지 데이터 추가

- ⊕ 학습 데이터의 개수를 직접 늘리면 더할 나위 없이 좋지만 이미지 데이터를 추가로 구하는 것은 사람의 손을 필히 거쳐야 하며 비용도 시간도 많이 드는 일임
- ⊕ 비용과 시간도 적게 소요하면서 학습 데이터의 다양성을 늘릴 수 있는 방법으로 고안된 것이 **데이터 증강 기법**
 - └ 하나의 원본 이미지를 다양한 버전으로 만들어 학습시키는 것



데이터 증강(Data Augmentation)



여러 가지 데이터 증강 기법

대표적 데이터 증강 기법

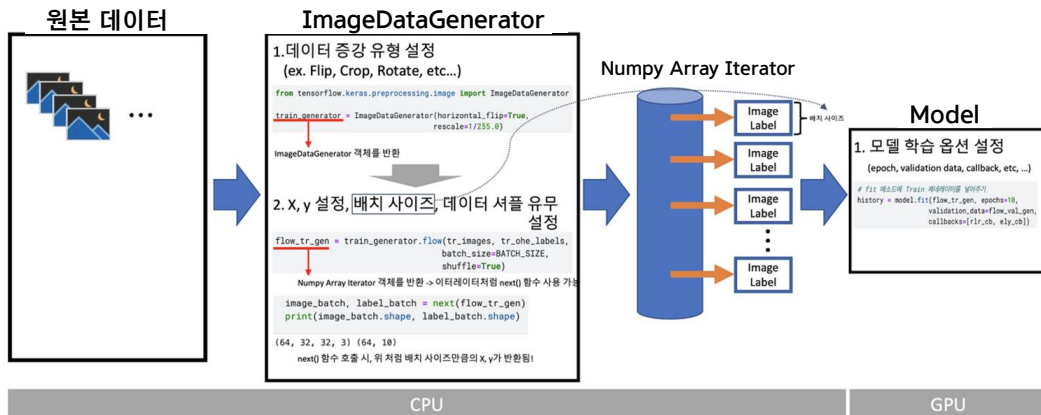
- Flipping : 원본 이미지를 수평 또는 수직으로 반전시키는 방법
- Rotating : 회전시키는 방법
- Cropping : 일부를 자르는 방법
- Colour Jittering : RGB 채널 순서를 바꾸거나 픽셀값을 변경하여 원본 이미지의 색조를 변경시키는 방법

데이터 증강(Data Augmentation)

텐스플로에서는 원본 이미지 증강 기법을 tensorflow.keras의 **ImageDataGenerator** 클래스가 대신함

ImageDataGenerator 클래스의 역할

- 원본 이미지를 증강
- 텐스플로 내에서 하나의 데이터 파이프라인을 구축
- Tensorflow.keras의 preprocessing 모듈에 존재



ImageDataGenerator 클래스의 과정

※ ImageDataGeneratorExam.ipynb
프로그램 실습을 통해
좀 더 깊이 있게 학습해 보자.

좀 더 구체적으로 ImageDataGenerator 과정을 정리해 보면 다음과 같다.

1 ImageDataGenerator 객체를 생성하면서 데이터 증강을 수행할 유형들을 지정함

```
1 # ImageDataGenerator 클래스를 통해 train_datagen 인스턴스를 만듭니다.  
2 train_datagen = ImageDataGenerator(rescale=1./255,      # 픽셀 값을 0~1 범위로 변환  
3                                   rotation_range=15,      # 15도까지 회전  
4                                   shear_range=0.2,        # 20%까지 기울임  
5                                   zoom_range=0.2          # 20%까지 확대  
6                                   )
```

🕒 위 코드와 같이 다양한 데이터 증강 기법을 설정하는 것을 볼 수 있음

좀 더 구체적으로 `ImageDataGenerator` 과정을 정리해 보면 다음과 같다.

2 1단계에서 생성한 ImageDataGenerator 객체를 생성한 후 훈련 데이터에서 배치 크기 만큼 이미지를 만들어 냄

- 🟢 학습 이미지(X), 정답 레이블(y) 이미지의 **배치 사이즈** 설정
- 🟢 **데이터 셔플** 여부 지정
- 🟢 반환되는 **레이블** 배열의 **유형** 지정
- 🟢 Numpy **Array Iterator**(iterator 안의 데이터를 하나씩 호출)

```
1 # train_datagen 인스턴스를 통해 trin dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 training_set 객체를 생성한다.  
2 training_set = train_datagen.flow_from_directory('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/train',  
3 |         target_size=(image_width, image_height),  
4 |         batch_size=batch_size,  
5 |         class_mode='binary')
```

- 위와 같이 설정하는 이유는 이미지 사이즈가 커질수록 한번에 이미지를 CPU, GPU에 올릴 수 없기 때문에 배치 사이즈 크기 설정 같은 처리를 미리 수행함

좀 더 구체적으로 `ImageDataGenerator` 과정을 정리해 보면 다음과 같다.

3 모델을 학습시키기 위해 필요한 하이퍼파라미터를 설정함

- 2단계에서 만들어진 `iterator`를 생성된 모델의 메서드(`fit` 또는 `fit_generator`)에 입력
- 에폭(반복 학습 횟수) 설정
- 콜백함수(모델이 더 이상 학습을 못할 경우 종료) 설정

```
1 # model training(모델을 학습시킵니다)
2 history = model.fit_generator(training_set,
3                               steps_per_epoch=512//batch_size,
4                               epochs=no_of_epochs,
5                               validation_data=test_set,
6                               validation_steps=64//batch_size,
7                               callbacks=[monitor_val_acc, modelCheckpoint])
8
```

④ 폐렴 진단(학습)

`train_datagen` 인스턴스를 통해 **훈련 데이터 셋**에서 한번에 `batch_size` 만큼 이미지를 만들어내는 `train_set` 객체를 생성한다.

- 🟢 train_datagen 인스턴스는 ImageDataGenerator 클래스의 객체

[illegible]

Found 5216 images belonging to 2 classes.

- flow_from_directory() 함수는 **directory** 경로로부터 데이터프레임을 가져오고, **증강(Augmented)** 또는 **정규화(Normalized)**된 **데이터 배치**를 생성함
- 위 코드에서 **훈련용 데이터** 셋에서 **batch_size=10** 만큼 이미지로 **train_set** 객체가 만들어진 것을 알 수 있음
- 실행 결과 훈련 데이터 셋으로 **2개의 클래스**로 구성된 **5,216장의 이미지**가 있음을 알 수 있음

⊕ 페럼 진단(학습)

ImageDataGenerator 클래스를 통해 테스트 데이터 셋(Test data set)에서 한번에 batch_size 만큼 이미지를 만들어내는 test_set 객체를 생성한다.

```
1 # val_datagen 인스턴스를 통해 test dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 test_set 객체를 생성한다.  
2 test_set = val_datagen.flow_from_directory('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/test',  
3                                           target_size=(image_width, image_height),  
4                                           batch_size=batch_size,  
5                                           class_mode='binary')
```

Found 624 images belonging to 2 classes.

- 위 코드에서 테스트 데이터 셋에서 batch_size = 10 만큼 이미지로 test_generator 객체가 만들어진 것을 알 수 있음
- 실행 결과 검증용 데이터 셋으로 2개의 클래스로 구성된 624장의 이미지가 있음을 알 수 있음

⊕ 페럼 진단(학습)

ImageDataGenerator 클래스를 통해 검증용 데이터 셋(validation data set)에서 한번에 batch_size 만큼 이미지를 만들어내는 val_set 객체를 생성한다.

```
1 # val_datagen 인스턴스를 통해 val dataset 에서 한번에 batch_size 만큼 이미지를 만들어내는 val_set 객체를 생성한다.  
2 val_set = val_datagen.flow_from_directory('gdrive/My Drive/CV/Diagnose Pneumonia/chest_xray/chest_xray/val',  
3                                         target_size=(image_width, image_height),  
4                                         batch_size=1,  
5                                         shuffle=False,  
6                                         class_mode='binary')
```

➤ Found 16 images belonging to 2 classes.

- 위 코드에서 검증용 데이터 셋에서 batch_size = 1 만큼 이미지로 val_set 객체가 만들어진 것을 알 수 있음
- 실행 결과 검증용 데이터 셋으로 2개의 클래스로 구성된 16장의 이미지가 있음을 알 수 있음

⊕ 페렴 진단(학습)

모델의 학습 시 가장 최근의 최고 모델을 저장하는 옵션을 정의한다.

```
1 # 콜백함수 정의
2 # 모델 학습시 가장 최신의 최고 모델을 저장하는 인스턴스를 정의
3
4 modelCheckpoint = ModelCheckpoint('diagnose_pneumonia_model.h5', save_best_only=True)
```

⊕ 페렴 진단(학습)

모델의 학습 시 정확도를 모니터링하여 조기에 학습을 종료시키는 옵션을 정의한다.



```
1 # 콜백함수 정의
2 # 모델 학습시 정확도를 모니터링하여 조기에 학습을 종료시키는 인스턴스 정의
3
4 monitor_val_acc = EarlyStopping(monitor='val_accuracy', patience=5)
```


정의된 인공지능망 모델을 학습시켜 보자.

```
▶ # model training(모델을 학습시킵니다)
history = model.fit_generator(training_set,
                              steps_per_epoch=5216//batch_size,
                              epochs=no_of_epochs,
                              validation_data=test_set,
                              validation_steps=624//batch_size,
                              callbacks=[monitor_val_acc, modelCheckpoint])
```

- ❶ 위 코드에서 모델 학습 중 callbacks = [monitor_val_acc]에 의해서 정확도를 모니터링하다가 증가되지 않고 떨어지거나 변화가 거의 없다면 patience = 5(위 예제의 경우) 만큼 기다리다 학습을 조기에 중단시킴
- ❷ 위 코드에서 모델 학습 중 callbacks = [modelCheckpoint]에 의해 모델 학습 중 가장 최근의 최고 모델을 VM에 diagnose_pneumonia_model.h5 파일 이름으로 저장함

정의된 인공지능망 모델을 학습시켜 보자.

```
Epoch 1/30
521/521 [=====] - 3351s 6s/step - loss: 0.5611 - accuracy: 0.7463 - val_loss: 0.7723 - val_accuracy: 0.6806
Epoch 2/30
521/521 [=====] - 113s 216ms/step - loss: 0.4598 - accuracy: 0.7912 - val_loss: 0.6071 - val_accuracy: 0.6516
Epoch 3/30
521/521 [=====] - 112s 215ms/step - loss: 0.3534 - accuracy: 0.8519 - val_loss: 0.3670 - val_accuracy: 0.8516
Epoch 4/30
521/521 [=====] - 113s 216ms/step - loss: 0.2649 - accuracy: 0.8897 - val_loss: 0.5600 - val_accuracy: 0.7210
Epoch 5/30
521/521 [=====] - 112s 215ms/step - loss: 0.2116 - accuracy: 0.9153 - val_loss: 0.3640 - val_accuracy: 0.8694
Epoch 6/30
521/521 [=====] - 112s 216ms/step - loss: 0.2126 - accuracy: 0.9164 - val_loss: 0.3376 - val_accuracy: 0.8694
Epoch 7/30
521/521 [=====] - 113s 217ms/step - loss: 0.1856 - accuracy: 0.9295 - val_loss: 0.3311 - val_accuracy: 0.8742
Epoch 8/30
521/521 [=====] - 112s 215ms/step - loss: 0.1709 - accuracy: 0.9326 - val_loss: 0.3441 - val_accuracy: 0.8726
Epoch 9/30
521/521 [=====] - 113s 217ms/step - loss: 0.1547 - accuracy: 0.9376 - val_loss: 0.2652 - val_accuracy: 0.9081
Epoch 10/30
521/521 [=====] - 114s 219ms/step - loss: 0.1615 - accuracy: 0.9418 - val_loss: 0.5438 - val_accuracy: 0.8323
Epoch 11/30
521/521 [=====] - 114s 218ms/step - loss: 0.1525 - accuracy: 0.9431 - val_loss: 0.4592 - val_accuracy: 0.8387
Epoch 12/30
521/521 [=====] - 114s 218ms/step - loss: 0.1400 - accuracy: 0.9502 - val_loss: 0.3756 - val_accuracy: 0.8790
Epoch 13/30
521/521 [=====] - 114s 219ms/step - loss: 0.1468 - accuracy: 0.9456 - val_loss: 0.2841 - val_accuracy: 0.9113
Epoch 14/30
521/521 [=====] - 113s 218ms/step - loss: 0.1377 - accuracy: 0.9497 - val_loss: 0.3599 - val_accuracy: 0.8694
Epoch 15/30
521/521 [=====] - 115s 221ms/step - loss: 0.1440 - accuracy: 0.9454 - val_loss: 0.3200 - val_accuracy: 0.9099
Epoch 16/30
521/521 [=====] - 114s 219ms/step - loss: 0.1384 - accuracy: 0.9474 - val_loss: 0.1946 - val_accuracy: 0.9258
Epoch 17/30
521/521 [=====] - 114s 219ms/step - loss: 0.1351 - accuracy: 0.9506 - val_loss: 0.3368 - val_accuracy: 0.9016
Epoch 18/30
521/521 [=====] - 115s 220ms/step - loss: 0.1324 - accuracy: 0.9522 - val_loss: 0.4300 - val_accuracy: 0.8742
Epoch 19/30
521/521 [=====] - 114s 218ms/step - loss: 0.1316 - accuracy: 0.9520 - val_loss: 0.3181 - val_accuracy: 0.9000
Epoch 20/30
521/521 [=====] - 115s 220ms/step - loss: 0.1315 - accuracy: 0.9531 - val_loss: 0.2688 - val_accuracy: 0.9194
Epoch 21/30
521/521 [=====] - 115s 220ms/step - loss: 0.1176 - accuracy: 0.9562 - val_loss: 0.3055 - val_accuracy: 0.9129
```

실행 결과 16epoch에서 정확도가 92.58%이고, 17~21epoch 학습을 진행하였지만
정확도가 개선되지 않아 학습이 완료된 것을 볼 수 있음

⊕ 페럼 진단(학습)

학습된 인공지능망 모델을 테스트 데이터 셋으로 평가해 보자.

```
▶ 1 # model evaluate(모델을 평가해 봅니다)
2 score = model.evaluate(test_set, steps=624//batch_size, verbose=1)
3 print('Val loss:', score[0])
4 print('Val accuracy:', score[1]*100)
```

```
↳ 62/62 [=====] - 6s 97ms/step - loss: 0.3686 - accuracy: 0.8548
Val loss: 0.3686389625072479
Val accuracy: 85.48387289047241
```

실행 결과 테스트 데이터 셋에 대한 정확도가 85.48% 나온 것을 볼 수 있음

⊕ 폐렴 진단(학습)

test_set이 만들어 내는 이미지의 클래스 인덱스를 확인해 보자.

```
1 # test_set 이 만들어 내는 이미지의 클래스 인덱스가 무엇인지 확인해 본다.  
2 print(test_set.class_indices)
```

```
{'NORMAL': 0, 'PNEUMONIA': 1}
```

- 실행 결과 정상인 경우는 0, 폐렴인 경우는 1임을 알 수 있음
- 위 코드에서 class_indices 객체 변수의 자료구조는 딕셔너리인 것을 알 수 있음

⊕ 페렴 진단(학습)

다음은 학습된 인공신경망 모델에 대해 훈련 데이터 셋과 검증용 데이터 셋의 손실값(Loss), 정확도(Accuracy)의 추이를 그래프로 표현해 보자.

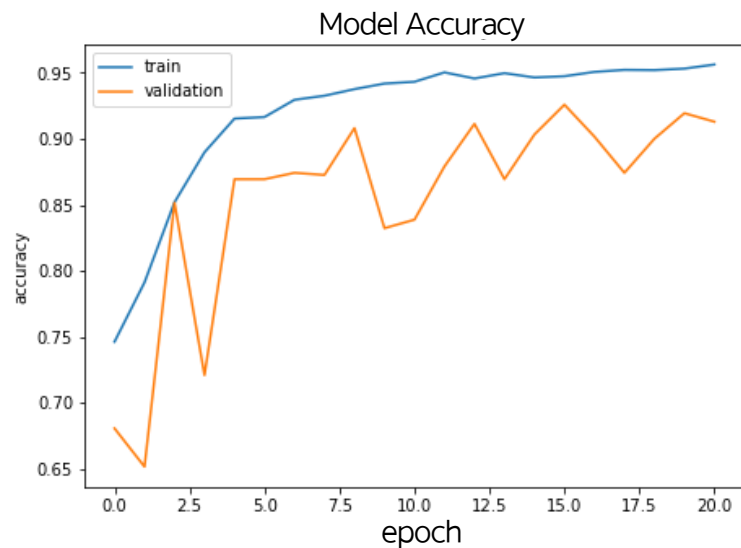
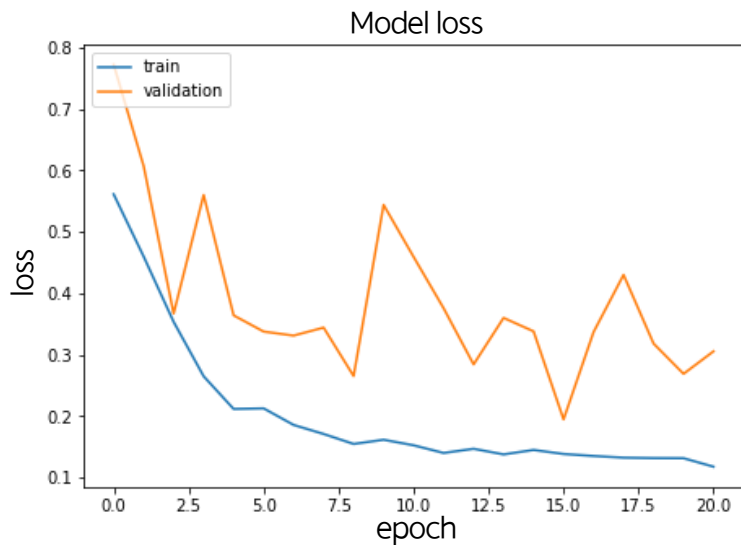
```
plt.figure(figsize=(16,5))

# 만들어진 모델에 대해 train dataset과 validation dataset의 loss 를 그래프로 표현해 봅니다.
plt.subplot(1, 2, 1)
plt.plot(history.history['loss'])
plt.plot(history.history['val_loss'])
plt.title('model loss')
plt.ylabel('loss')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')

# 만들어진 모델에 대해 train dataset과 validation dataset의 accuracy 를 그래프로 표현해 봅니다.
plt.subplot(1, 2, 2)
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])
plt.title('model accuracy')
plt.ylabel('accuracy')
plt.xlabel('epoch')
plt.legend(['train', 'validation'], loc='upper left')
```

페럼 진단(학습)

다음은 학습된 인공신경망 모델에 대해 **훈련 데이터 셋**과 **검증용 데이터 셋**의 **손실값(Loss)**, **정확도(Accuracy)**의 **추이**를 **그래프로** 표현해 보자.



- 훈련 데이터 셋의 경우 손실값이 점차 줄어 학습이 잘 진행되는 것으로 보임
- 훈련 데이터 셋의 경우 정확도가 점차 높아지고 있어 학습이 잘 진행되는 것으로 보임

⊕ 페럼 진단(학습)

학습된 모델에 `val_set` 객체가 만들어낸 이미지로 분류 예측을 수행해 보자.

🔄 여기서는 16장 이미지를 분류 예측하고 그 결과를 `predictions` 객체에 저장함



```
1 # 학습된 모델에 val_set 객체가 만들어낸 이미지로 분류 예측을 수행해 보자.  
2 predictions = model.predict(val_set, steps=16, verbose=1)
```



16/16 [=====] - 16s 560ms/step

⊕ 페렴 진단(학습)

학습된 모델에 `val_set` 객체가 만들어진 이미지로 분류 예측을 수행해 보자.

- 다음은 `predictions` 객체의 형상과 예측결과를 확인한 것임
- 예측결과 값의 분포는 0~1사이의 실수인 것을 알 수 있음
- 즉, 0 ~ 0.5 이하의 경우 'NORMAL', 0.5 보다 큰 값은 'PNEUMONIA'로 분류됨

```
1 # 예측 결과 객체 predictions shape 확인
2 predictions.shape
```

↳ (16, 1)

```
1 # 예측 결과 확인
2 predictions
```

↳ array([[0.7094597],
[0.5348588],
[0.23154846],
[0.83332115],
[0.98341817],
[0.9487347],
[0.3363725],
[0.02327376],
[0.99761],
[0.99450207],
[0.90449005],
[0.9086135],
[0.97572815],
[0.994443],
[0.98599225],
[0.98664063]], dtype=float32)

출력층 활성화 함수 : 시그모이드

0 ~ 0.5 이하 : NORMAL
0.5 보다 큰 값 : PNEUMONIA

다음은 학습된 모델이 예측한 16장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화한 것이다.

```
1 # Predict Result(val_set 가 만들어 낸 이미지를 우리가 만든 모델을 통해 예측한 결과를 보여줍니다.)
2 f,ax = plt.subplots(4,4,figsize=(20,20))
3
4 for i in range(len(predictions)):
5     image, label = val_set.next() # val_set 이 만들어 낸 이미지를 하나씩 꺼내온다
6     real_class = 'NORMAL' if int(label) == 0 else 'PNEUMONIA' # val_set 이 만들어 낸 이미지의 label name 을 가져옴
7     ax[i//4,i%4].imshow(image[0]) # val_set 이 만들어 낸 이미지를 보여줌
8     ax[i//4,i%4].axis('off')
9
10    pred_class = 'PNEUMONIA' if predictions[i] > 0.5 else 'NORMAL' # val_set 이 만들어 낸 이미지를 모델을 통해 예측한 결과 값의 label name
11    pred_prob = predictions[i] # val_set 이 만들어 낸 이미지를 모델을 통해 예측한 결과(확률값)
12    ax[i//4,i%4].set_title('REAL:' + real_class + '\nPREDICT:' + pred_class + '(' + str(pred_prob) + ')') # title 에 실제값과 모델에 의해
13
14 f.suptitle('Predict Result')
15
16 plt.show()
```

위 코드에서 predictions 객체의 원소 값이 0.5이하이면 'NORMAL', 0.5보다 크면
'PNEUMONIA'로 예측한 것으로 판단하고 있음

다음은 학습된 모델이 예측한 16장 이미지의
예측결과(predictions 객체에 저장된 결과)를 시각화한 것이다.

예측결과를 보면 모델은 정확도 75%로 분류한 것을 알 수 있음

정답 : 12
오답 : 4

